COMPUTER ENGINEERING

QUEZON CITY

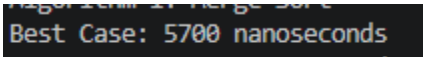
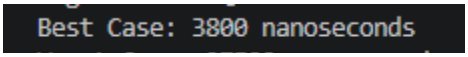
COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
NAME:	Condino, Niel Vincent B. Guarin, Jan Mikaela C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14 **Algorithm Complexity**

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Merge Sort	O (n log n)	O (n log n)	In merge sort, the **best case** scenario occurs when the data is already sorted. This algorithm compares elements until a smaller subarray runs out completely. On the other hand, the **worst case** scenario occurs when the elements in an array are completely alternating.
Quick Sort	O (n log n)	O (n^2)	Quick sort arranges elements in order by choosing one element which is the pivot, dividing the elements into two groups. The **best case** occurs when the pivot divides the array into two equal parts. The **worst case** happens when the pivot creates unbalanced partitions, where one sub - array contains n-1 elements and the other contains 0 elements.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Merge Sort	Quick Sort
Best Case		
Worst Case		
Analysis	Merge sort has the same best and worst case time complexity. After executing the program, the difference between the two results was relatively small, showing that Merge Sort performed consistently regardless of the input arrangement. Merge	Quick sort was relatively faster than Merge Sort in the best case, but its execution time increased significantly in the worst case. This happens because the pivot can create unbalanced partitions, causing Quick Sort to perform more comparisons and take longer to

	Sort divides the array into smaller parts and merges them in sorted order.	sort the array.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A. Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

Pseudocode Analysis

Comparing the two, the clear winner is the binary search. It assumes the array is sorted which it then leverages this feature; it checks for the middle of the given data storage, then checks the left or right side of the data storage, effectively cutting the search space in half every iteration. This is what gives it the logarithmic runtime. The linear search on the other hand checks then compares each element to the target, however the search time scales with the search space; giving it a linear runtime. This is advisable when the search space is small or there is no way to sort the elements of the given data storage.

Time and Space Complexity

As discussed above, the linear search checks every element of the array for its intended target. As the input size grows, so is the search space and the runtime. Because the runtime scales linearly with the input, the algorithm has a **Linear runtime** or can be expressed as **O(n)**. Binary search effectively cuts halves of the array each iteration. Because of this behaviour, the runtime grows **logarithmically** or slowly with the input size or the runtime can be expressed as **O(log n)**. The space complexity for both of these functions or algorithms is **O(1)** or constant. As the input data scales with the size, the functions memory remains the same.

Table 14-3: Experimental Analysis of Worst time complexity of each algorithm

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	`Worst Case Binary Search Algorithm's Time: 300 nanoseconds`	`Worst Case Linear Search Algorithm's Time: 2800 nanoseconds`
10,000	`Worst Case Binary Search Algorithm's Time: 500 nanoseconds`	`Worst Case Linear Search Algorithm's Time: 22000 nanoseconds`
100,000	`Worst Case Binary Search Algorithm's Time: 900 nanoseconds`	`Worst Case Linear Search Algorithm's Time: 228100 nanoseconds`

With the theoretical analysis in mind, it is proven that as the input size grows, the runtime for linear search scales with this increase in input size. From ~2000 to ~20,000 to ~200,000 at each increase in input size. With a 10x increase in input size, there is a 10x increase in runtime. However, looking at binary search, as the input size grows, the runtime grows slowly. There is just around less than 2x increase for each of the 10x increase in input size. This proves the theoretical analysis is relatively true.

CONCLUSION

Condino: In this activity, I am able to apply my learning about runtime complexities into my old algorithms. I was able to test and see how the runtime is affected by the input size. I observed that using theoretical analysis, I can find the relative growth of runtime in relation to the input size. While there are many factors that affect an algorithm's runtime, we can use theoretical analysis to provide a look of how it will scale with the input. Giving us advice whether we need to optimize the code because of how it scales. Overall, this lesson about runtime complexity helped me with further understanding and knowing what algorithm to apply based on the problem. I plan to use this everytime I come up with an algorithm to keep track of the efficiency of my algorithm.

Guarin: In this activity, I learned how to analyze the time complexity of algorithms and compare their performance using best-case and worst-case scenarios. I observed that Merge Sort had more consistent execution times, while Quick Sort performed faster in the best case but took longer in the worst case. I also learned that the input arrangement can affect Quick Sort's performance because it uses a pivot to divide the array. This activity helped me understand why analyzing algorithms is important when choosing an efficient sorting method. I can improve my work by testing more input sizes and repeating the experiments to get more reliable results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.